



Est. 2000

ABES Engineering College

INDEX

Exp. No.	Name of the Experiment	Date of Practical	Page No. From _____ To _____	Pre-Lab Writing Work MM: 0
		Scheduled / Actual		
01	BREADTH FIRST SEARCH	22/08/2025 22/08/2025	01	✓
02	WATER JUG PROBLEM	29/08/2025 29/08/2025	02 to 05	✓
03	PUNCTUATION REMOVAL	12/09/2025 12/09/2025	06	✓
04	SORT THE SENTENCE	19/09/2025 19/09/2025	07	✓
05	TIC TAC TOE	03/10/2025 03/10/2025	08 to 15	✓
06	HANGMAN GAME	17/10/2025 17/10/2025	16 to 19	✓
07	REMOVE STOP WORDS	07/11/2025 07/11/2025	20 to 21	
08	STEMMING USING NLTK		22	
09	POS TAGGING USING NLTK		23	
10	LEMMAIZATION USING NLTK		24	
11	TEXT CLASSIFICATION USING NLTK		25 to 26	
12	N-QUEENS PROBLEM		27 to 28	
13	SPEECH-TO-TEXT & TEXT-TO-SPEECH		29 to 30	

Teacher's Remarks (if any):

Name & Sign. of Faculty member(s) with date

Name & Sign.

Notes: 1. *Pre-Lab writing

AIM:- Write a python program to implement stemming for a given sentence using NLTK

CODE:-

```
from nltk.stem import PorterStemmer
from nltk.tokenize import
word_tokenize ps = PorterStemmer()
example_words = ["python, pythonly, phythonez,
                  pythonly"]
for w in example_words:
    print(ps.stem(w))
```

OUTPUT:

python, pythonly, phythonez, pythonli

AIM: Write a python program to POS (Parts of Speech) tagging for the given sentence using NLTK.

CODE:

```
import nltk
text = nltk.word_tokenize('ive into NLTK: part-of-  
speech tagging and POS Tagger')
pos = nltk.pos_tag(text)
print(text)
print(pos)
```

OUTPUT:

```
['ive', 'into', 'NLTK', ':', 'Part-of-speech', 'tagging', 'and',  
'POS', 'Tagger']
```

```
[('ive', 'JJ'), ('into', 'IN'), ('NLTK', 'NNP'), (':', ':'),  
(('Part-of-speech', 'JJ'), ('tagging', 'NN'), ('and', 'CC'),  
(('POS', 'NNP'), ('Tagger', 'NNP'))]
```

AIM: Write a python program to implement Lemmatization using NLTK.

CODE:

```
from nltk.stem import  
WordNetLemmatizer lemmatizer =  
WordNetLemmatizer()  
print(lemmatizer.lemmatize("cats"))  
print(lemmatizer.lemmatize("cacti"))  
print(lemmatizer.lemmatize("geese"))  
print(lemmatizer.lemmatize("rocks"))  
print(lemmatizer.lemmatize("pyth"))  
print(lemmatizer.lemmatize("cats"))
```

Output:

cat cactusgooserock pyth cat

ATM: Write a python program to for Text Classification for the given sentence using NLTK.

CODE:

```
import nltk import random
from nltk.corpus import movie_reviews
documents = [(list(movie_reviews.words(fileid)),
                category)]
for category in movie_reviews.categories():
    for fileid in movie_reviews.fileids(category):
        random.shuffle(documents)
print(documents[1]) all-words = []
for w in movie_reviews.words():
all-words = nltk.FreqDist(all-words

all-words = []
for w in movie_reviews.words():
    all-words.append(w.lower())
all-words = nltk.FreqDist(all-words)
print(all-words.most-common(15))
print(all-words["stupid"])
```

Out put:

([u'making', u'you', u'first', u'feature', u'film',
u'ain', u't', u'easy', u', u'assemble', u'a',
u'decent', u', u'if', u'not', u', u'strong', u'cast', u',
u'as', u'writer', u', u'director', u'robert', u'moresco',
u'has', u'done', u'with', u'one', u'eyed', u'king', u',
u'and', u'you', u'', u're', u'already', u'ahead', u'of',
u'the', u'game', u', u'but', u'rehash', u'old',
u'plot', u'lines', u', u'tired', u'dialogue', u', u'and',

u'standard', u'clich', u'?', u's', u'?', u'and', u'a',
 u'well', u'-'', u'intentioned', u'effort', u'such', u'as',
 u'this', u'one', u'could', u'jeopardize', u'your', u'chance',
 u'at', u'a', u'second', u'feature', u'film', u'.', u'how',
 u'many', u'more', u'movies', u'do', u'we', u'need', u'about',
 u'a', u'rough', u'neighborhood', u'full', u'of', u'lifelong',
 u'friends', u'hopelessly', u'turned', u'to', u'crime',
 u'or', u'worse', u'.', u'the', u'enormous', u'catalog',
 u'of', u'such', u'movies', u'might', u'dissuade', u'ad',
 u'filmmaker', u'from', u'making', u'yet', u'another', u'.',
 u'but', u'here', u'we', u'have', u'it', u'.', u'again', u'.',
 u'five', u'wish', u'kids', u'in', u'ngc', u'', u's', u'hell',
 u'', u's', u'kitchen', u'make', u'an', u'overemotional',
 u'part', u'over', u'some', u'stolen', u'rings', u'on', u'an',
 u'anonymus', u'and', u'slow', u'motion', u'.', u'in',
 u'the', u'film', u'', u's', u'first', u'scene', u'.', u'neg')

AIM: Write a program to implement N-Queen Problem.

CODE:

```
global N
N=4
```

```
def printSolution(board):
    for i in range(N):
        for j in range(N):
            print(board[i][j], end = ' ')
        print()
```

```
def isSafe(board, row, col):
    # Check upper left diagonal
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == 1:
            return False
```

```
# for i, j in zip(range
# Check upper left diagonal.
for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
```

```
# Check lower left diagonal
for i, j in zip(range(row, N, 1), range(col, -1, -1)):
    if board[i][j] == 1:
        return False.
```

```
# Check left column
for i in range(row+1):
    if board[i][col] == 1:
        return False
```

```
return True.
```

N-Queens Problem (Backtracking) Practical No. 12

Practical Name:

```
def solveNQueensUtil(board, col):
    if col >= N:
        return True
```

```
    for i in range(N):
        if isSafe(board, i, col):
            board[i][col] = 1
```

```
        if solveNQueensUtil(board, col + 1):
            return True
```

```
        board[i][col] = 0 # Backtrack
    return False.
```

```
def solveNQueens():
    board = [[0 for _ in range(N)] for _ in range(N)]
    for _ in range(N)]
```

```
    if solveNQueensUtil(board, 0) == False:
        print("Solution does not exist")
        return False
```

```
    printSolution(board)
    return True
```

solveNQueens()

OUTPUT:

```
0 1 0 0
1 0 0 1
0 0 1 0
0 0 0 1
```

AIM: Write a Python program to translate speech to text and text to speech.

CODE:

```
import speech_recognition as sr
import pyttsx3
```

```
# Initialize speech recognizer and TTS engine.
recognizer = sr.Recognizer()
engine = pyttsx3.init()
```

```
def speech-to-text():
```

```
    with sr.Microphone() as source:
```

```
        print("Say something...")
```

```
        audio = recognizer.listen(source)
```

```
        try:
```

```
            text = recognizer.recognize_google(audio)
```

```
            print(f"You said: {text}")
```

```
            return text
```

```
        except sr.UnknownValueError:
```

```
            print("Could not understand audio")
```

```
        except sr.RequestError as e:
```

```
            print(f"Error: {e}")
```

```
def text-to-speech(text):
```

```
    engine.say(text)
```

```
    engine.runAndWait()
```

```
if __name__ == "__main__":
```

```
    # Speech to Text
```

```
    spoken_text = speech-to-text()
```

```
if spoken-text:
    print("Converting to speech...")
    text-to-speech(spoken-text)
```

OUTPUT:

Say something...

You said: Hello this is a test.

Converting to speech... (audio playback)